



Atmosphere System Documentation

Scope: Complete documentation of the ISA atmosphere model and speed of sound calculations used throughout the climb trajectory simulation system.

Table of Contents

- [Atmosphere System Documentation](#)
 - [Table of Contents](#)
 - [1\) System Overview and Objectives](#)
 - [2\) Mathematical Foundation](#)
 - [2.1 ISA Temperature Model](#)
 - [2.2 Speed of Sound Calculation](#)
 - [3\) Code Implementation](#)
 - [3.1 Atmospheric Properties Function](#)
 - [3.1.1 ISA Properties Calculation Flow](#)
 - [3.2 Speed of Sound Function](#)
 - [3.2.1 Speed of Sound Calculation Flow](#)
 - [4\) Integration with Aircraft Systems](#)
 - [4.1 Engine Performance Integration](#)
 - [4.2 Aerodynamic Integration](#)
-

1) System Overview and Objectives

The atmosphere system provides fundamental atmospheric properties required for aircraft performance calculations. All altitude inputs are in **meters**, and the system implements the International Standard Atmosphere (ISA) model with proper temperature lapse rate handling.

Key Components:

- ISA temperature, pressure, and density calculations

- Speed of sound computation with temperature dependency
 - Integration with engine performance and aerodynamic calculations
-

2) Mathematical Foundation

2.1 ISA Temperature Model

Theory: The International Standard Atmosphere defines temperature as a function of altitude with a linear lapse rate in the troposphere and constant temperature in the stratosphere.

Mathematical Formulation:

$$T(h) = \begin{cases} T_0 + L \cdot h, & h \leq 11,000 \text{ m} \\ T(11,000), & h > 11,000 \text{ m} \end{cases}$$

Where:

- $T_0 = 288.15$ K (sea level temperature)
- $L = -0.0065$ K/m (temperature lapse rate)
- h = altitude in meters

Physical Interpretation:

- **Troposphere (0-11 km):** Linear temperature decrease with altitude
- **Stratosphere (>11 km):** Constant temperature (216.65 K)
- **Temperature gradient:** -6.5°C per 1000m altitude gain

2.2 Speed of Sound Calculation

Theory: Speed of sound in air depends on temperature and atmospheric composition.

Mathematical Formulation:

$$a(T) = \sqrt{\gamma R T}$$

Where:

- $\gamma = 1.4$ (ratio of specific heats for air)
- $R = 287.05$ J/(kg·K) (specific gas constant for air)
- T = absolute temperature (K)

Physical Interpretation:

- Speed of sound decreases with altitude due to temperature decrease
 - Critical for Mach number calculations: $M = V/a(T)$
 - Affects engine performance and aerodynamic characteristics
-

3) Code Implementation

3.1 Atmospheric Properties Function

Function Signature:

```
def isa_properties(h_m: float) -> tuple[float, float, float]:
    """
    Calculate ISA atmospheric properties at given altitude.

    Args:
        h_m: Altitude in meters

    Returns:
        tuple: (temperature_K, pressure_Pa, density_kg_m3)
    """
    return _atmospheric_properties.isa_properties(h_m)

def a_from_altitude(h_m: float) -> float:
    """Speed of sound from altitude using centralized atmosphere model."""
    return _atmospheric_properties.a_from_altitude(h_m)
```

Actual Implementation in Atmosphere Class:

```
class Atmosphere:
    def calculate_atmospheric_properties_meters(self, altitude_m: float):
        """Calculate atmospheric properties directly from altitude in meters."""
        return self._calculate_properties_from_meters(altitude_m)

    def get_speed_of_sound(self, altitude_m: float) -> float:
        """Compute speed of sound using T(h)."""
        T = self.get_temperature(altitude_m)
        return math.sqrt(1.4 * self.R * T)
```

3.1.1 ISA Properties Calculation Flow

Complete Logical Flow:

ISA PROPERTIES CALCULATION FLOW

START: isa_properties(h_m)



Step 1: Altitude Classification

└ IF $h_m \leq 11000$:

└ REGION: Troposphere

└ CONTINUE to troposphere calculation

└ ELSE:

└ REGION: Stratosphere

└ CONTINUE to stratosphere calculation



Step 2A: Troposphere Calculation

└ Temperature: $T = T_0 + L \times h_m$

└ $T_0 = 288.15$ K (sea level)

└ $L = -0.0065$ K/m (lapse rate)

└ Pressure: $P = P_0 \times (T/T_0)^{-g/(L \times R)}$

└ $P_0 = 101325.0$ Pa (sea level)

└ $g = 9.80665$ m/s²

└ $R = 287.05$ J/(kg·K)

└ Density: $\rho = P / (R \times T)$



Step 2B: Stratosphere Calculation

└ Temperature: $T = T_0 + L \times 11000$ (constant)

└ $T = 216.65$ K

└ Pressure: $P = P_{\text{trop}} \times \exp(-g \times (h_m - 11000) / (R \times T))$

└ P_{trop} = pressure at 11 km

└ Exponential decay with altitude

└ Density: $\rho = P / (R \times T)$



```
|   Step 3: Return Results   |  
| └─ RETURN: (T, P, rho) ─ |  
|_____|  
  
↓  
END: Return atmospheric properties
```

Implementation Details:

```
def isa_properties(h_m):  
    # ISA constants  
    T0 = 288.15 # Sea level temperature (K)  
    P0 = 101325.0 # Sea level pressure (Pa)  
    L = -0.0065 # Temperature lapse rate (K/m)  
    g = 9.80665 # Gravitational acceleration (m/s²)  
    R = 287.05 # Specific gas constant (J/(kg·K))  
  
    if h_m <= 11000:  
        # Troposphere: linear temperature decrease  
        T = T0 + L * h_m  
        P = P0 * (T / T0) ** (-g / (L * R))  
    else:  
        # Stratosphere: constant temperature  
        T = T0 + L * 11000 # 216.65 K  
        P = P0 * (T / T0) ** (-g / (L * R)) * np.exp(-g * (h_m - 11000) / (R * T))  
  
    # Density from ideal gas law  
    rho = P / (R * T)  
  
    return T, P, rho
```

3.2 Speed of Sound Function

Function Signature:

```
def a_from_T(T: float) -> float:
    """
    Calculate speed of sound from temperature.

    Args:
        T: Temperature in Kelvin

    Returns:
        float: Speed of sound in m/s
    """
```

Implementation:

```
def a_from_T(T):
    return float(np.sqrt(1.4 * 287.05 * T))
```

3.2.1 Speed of Sound Calculation Flow

Complete Logical Flow:

SPEED OF SOUND CALCULATION FLOW

START: a_from_T(T)



Step 1: Input Validation

└─ Validate temperature is positive

└─ Validate temperature is finite

└─ CONTINUE



Step 2: Speed of Sound Calculation

└─ Apply formula: $a = \sqrt{\gamma \times R \times T}$

└─ $\gamma = 1.4$ (ratio of specific heats)

└─ $R = 287.05 \text{ J/(kg}\cdot\text{K)}$ (gas constant)

└─ $T = \text{temperature (K)}$



Step 3: Return Result

└─ Convert to float for precision

└─ RETURN: speed_of_sound (m/s)



END: Return speed of sound

4) Integration with Aircraft Systems

4.1 Engine Performance Integration

Usage Pattern:


```
# Get atmospheric conditions
T, P, rho = isa_properties(altitude_m)

# Calculate speed of sound
a = a_from_T(T)

# Convert Mach to true airspeed
V = M * a

# Engine queries use altitude in meters
thrust = engine.thrust_with_lever(lever, M, altitude_m)
```

Key Integration Points:

- Engine performance tables indexed by altitude (meters)
- Mach number calculations for engine envelope compliance
- Temperature effects on engine performance

4.2 Aerodynamic Integration

Usage Pattern:

```
# Atmospheric properties for aerodynamic calculations
T, P, rho = isa_properties(altitude_m)
a = a_from_T(T)

# Mach number for drag calculations
M = V / a

# Density for lift calculations
L = 0.5 * rho * V**2 * S * CL
```

Key Integration Points:

- Density effects on aerodynamic forces
- Mach number for compressibility corrections
- Temperature effects on air properties